

Database keys

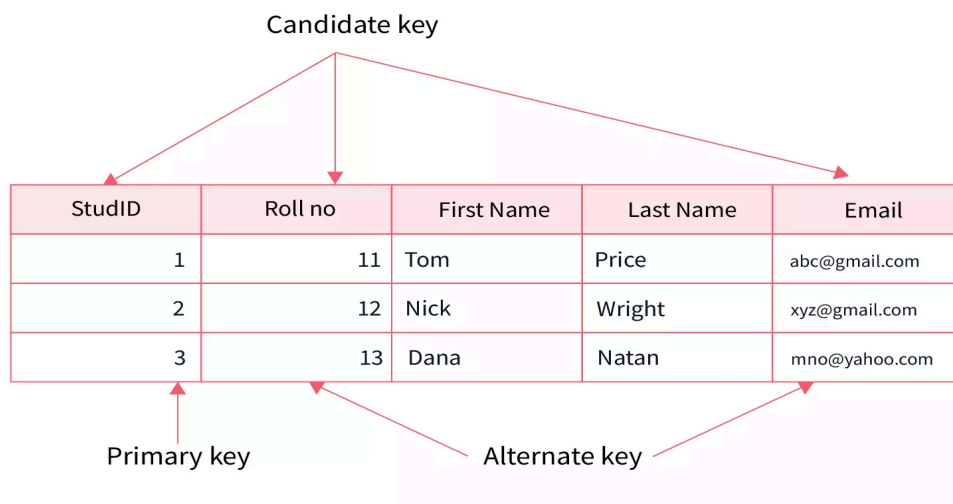
Keys are one of the basic requirements of a relational database model. It is widely used to identify the tuples(rows) uniquely in the table. We also use keys to set up relations amongst various columns and tables of a relational database.

Different Types of Database Keys

- Candidate Key
- Primary Key
- Super Key
- Alternate Key
- Foreign Key
- Composite Key

Primary Key:

- A primary key is a **unique identifier** for each record in a table.
- It cannot contain NULL values and must be unique for each record.
- Only one primary key can exist per table.



Candidate Key:

- A candidate key is a column or a set of columns in a table that uniquely identifies each row in the table.
- There can be multiple candidate keys in a table.
- From the set of candidate keys, one is chosen as the primary key.

Alternate Key:

- An alternate key, also known as a secondary key, is any candidate key that is not selected as the primary key.
- While it uniquely identifies rows, it is not chosen as the primary means of identification.

Super Key:

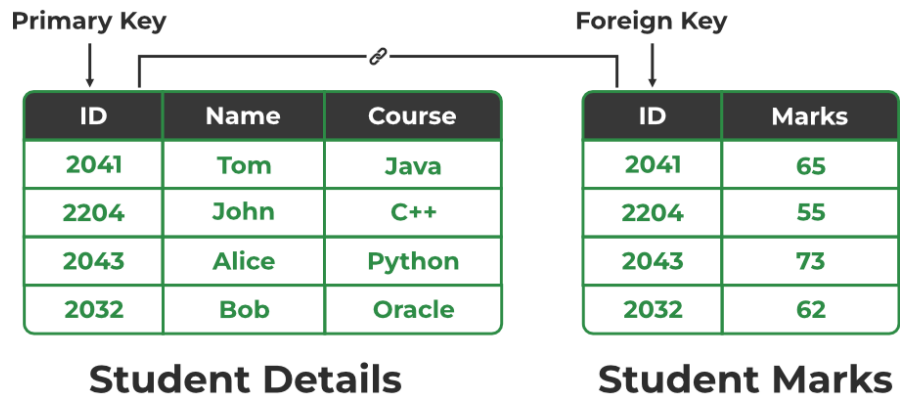
- A super key is a set of one or more columns that can uniquely identify each row in a table.
- It may contain more columns than necessary for uniqueness.
- Any subset of a super key is also a super key.



Foreign Key:

- A foreign key column in a table points to a column with unique values in another table (often the primary key column) to create a way of cross-referencing the two tables.
- It establishes a relationship between two tables.

- It ensures referential integrity by enforcing a link between the data in two tables.



Composite Key:

- A composite key is a key that consists of two or more columns in a table.
- Together, these columns uniquely identify a record in the table.
- Unlike primary keys, composite keys can contain NULL values, but their combination must be unique.

Composite Key			
Roll No.	Name	Age	Phone
1	Aryan	21	7491901521
2	Sachin	25	870904365
3	Prince	20	784600652
4	Anuj	21	9876534523

Data types in SQL

An SQL developer must decide what type of data that will be stored inside each column when creating a table. The data type is a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.

**In MySQL there are three main data types:
numeric, string, and date and time.**

Numeric Data types

- TINYINT(size) - A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The size parameter specifies the maximum display width (which is 255)

#It's not a negative physical dimension, but rather a description of the range of values that the data type can hold.

- SMALLINT() - A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The size parameter specifies the maximum display width (which is 255)

- MEDIUMINT(size) - A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The size parameter specifies the maximum display width (which is 255)

- INT(size) - A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The size parameter specifies the maximum display width (which is 255)

- BIGINT(size) - A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The size parameter specifies the maximum display width (which is 255)

Data Type	Details
tinyint	8 bits (0 to 255)
smallint	16 bit integer (-32768 to 32767)
int	32 bit integer (-2147483648 to 2147483647)
bigint	64 bit integer (-2^{63} to $2^{63} - 1$)
bit	Values of 1, 0

Numeric Data types

- FLOAT(size, d) - A floating point number. The total number of digits is specified in size.

The number of digits after the decimal point is specified in the d parameter.

- DOUBLE(size, d) - A normal-size floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter

- DECIMAL(size, d) - The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. The

maximum number for size is 65. The maximum number for d is 30. The default value for size is 10. The default value for d is 0.

String Data types

- **CHAR(size)** - A **FIXED** length string (can contain letters, numbers, and special characters). The size parameter specifies the column length in characters - can be from 0 to 255. Default is 1

- `CHAR` is a fixed-length character data type.
- When you define a column as `CHAR`, you specify a fixed length for the column, and each value stored in that column will occupy exactly that amount of space, padding with spaces if necessary.
- If you store a shorter string than the specified length, it will be padded with spaces to match the defined length.
- It's useful when you have data with a consistent length, such as country codes, where each code is a fixed number of characters.

Example:

```
CREATE TABLE student (  
  student_id INT PRIMARY KEY,  
  first_name CHAR(10),  
  last_name CHAR(10)  
);
```

In this example, both `first_name` and `last_name` columns are defined as `CHAR(10)`, meaning they can store strings of up to 10 characters in length. If you insert a name shorter than 10 characters, it will be padded with spaces to fill the remaining space.

- `VARCHAR(size)` - A VARIABLE length string (can contain letters, numbers, and special characters). The size parameter specifies the maximum string length in characters - can be from 0 to 65535.

`VARCHAR(10)`

- `CHAR` is a fixed-length character data type in SQL, which always reserves a specific amount of storage space, padding with spaces if necessary.

`VARCHAR`, on the other hand, is a variable-length character data type, which only uses as much storage as needed for the actual data, resulting in more efficient storage usage for shorter strings

`TEXT(size)` - Holds a string with a maximum length of 65,535 bytes

- `BLOB(size)` -

A `BLOB` (Binary Large Object) is a data type used to store large binary data, such as images, videos, audio files

For `BLOBs` (Binary Large Objects). Holds up to 65,535 bytes of data

- `TINYBLOB` - For `BLOBs` (Binary Large Objects). Max length: 255 bytes

- `TINYTEXT` - Holds a string with a maximum length of 255 characters

- `MEDIUMTEXT` - Holds a string with a maximum length of 16,777,215 characters

- `MEDIUMBLOB` - For `BLOBs` (Binary Large Objects). Holds up to 16,777,215 bytes of data

- LONGTEXT - Holds a string with a maximum length of 4,294,967,295 characters

- LONGBLOB - For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
- ENUM(val1, val2, val3, ...) - A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted.

Data types in SQL

DATE - A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'

- DATETIME - A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'.

- TIMESTAMP - A timestamp. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC

- TIME - A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'

- YEAR - A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000.

Char(10) hari - - - - -

varchar(10) hari